

Supplementary Materials

Simulation-informed transfer learning improves low-data nanobody thermal stability prediction

Taihei Murakami, Kentaro Sasaki, Soichiro Oda, Kazuma Okada, and Yasuhiro Matsunaga

Supplementary methods

Experimental target data and target-label scaling

All target-task comparisons use the same NbBench thermo-seq endpoint [1]. The processed tables hold an amino-acid sequence field and a melting-temperature label in degrees Celsius. The split sizes are 57 training, 114 validation, and 396 held-out test sequences. As described in Methods, we deliberately assigned the smallest NbBench partition to training and the largest to the test set so that the model sits in the low-data regime the study targets.

Split definitions. The splits are fixed throughout. The 57 training examples fit network parameters, the validation split drives checkpoint selection, early stopping, and choice among candidate settings, and the test split stays untouched until those choices are fixed. The same rule applies to Tm-only models and to models trained with computational labels.

Target-label scaling. Before optimization, the training workflow maps the experimental Tm values to a scaled regression target. The scaler is fit only on the 57 training labels, using robust scaling followed by linear rescaling to [0,1]. Validation labels are transformed with this training-fitted scaler during model selection. Test predictions are averaged across seeds in scaled-label space and inverse-transformed to degrees Celsius before errors are computed, so the validation and test label distributions never influence the fitted transform.

Experimental-label scaling reference. This reference uses the same training split but subsamples it to the requested number of labels with the run seed; the validation and test splits are never subsampled. It serves only as a positive control for the pipeline. Adding true target labels should lower the Tm prediction error.

Processed computational-label tables

Computational labels are loaded as computational tasks rather than appended to the Tm data as extra measurements. Table 1 lists the processed target and computational-label files used in the reported comparisons. Mutation-effect tables carry sequence fields and [0,1]-scaled computational-label fields. The MD native-contact tables use the same processed format so that they can enter the same computational-task loader; there, the scaled label is a [0,1]-scaled native-contact fraction rather than a mutation free energy. The FEP-matched MD table (the primary MD label) is built on the same 1MEL/4IDL mutation scans as FEP; the diverse-panel MD table provides the heterogeneous simulation-acquisition comparison.

Table 1 Processed target and computational-label data. Row counts exclude header rows.

Data type	Role	Processed rows	Use in reported comparisons
Experimental Tm	Target train	57	Model fitting
Experimental Tm	Target validation	114	Checkpoint and setting selection
Experimental Tm	Target test	396	Final held-out evaluation
FEP mutation free energy	Computational tasks	435 + 409	Main computational task
MD native-contact (FEP-matched scan)	Computational tasks	431 + 406	Matched-design MD label (primary)
Rosetta mutation score	Computational tasks	435 + 409	Computational-label comparison
ThermoMPNN stability score	Computational tasks	435 + 409	Computational-label comparison
Random variants scored by Rosetta	Computational tasks	1000 + 1000	Computational-label comparison
ESM2-proposed variants scored by Rosetta	Computational tasks	1000 + 1000	Computational-label comparison
MD native-contact (diverse panel)	Computational task	1143	Heterogeneous simulation-design comparison

Computational-task assignment. The loader assigns each row to a target or computational task. Experimental Tm rows use the Tm head. In the main mutation-effect model, the two template-specific tables use separate computational heads, and the MD-derived Q-value table uses a single computational head. This assignment fixes which regression head an example uses and which task-specific loss term it contributes to.

Sequence-overlap check. We checked exact sequence overlap between the processed computational tables and the fixed NbBench splits. The FEP, Rosetta, and ThermoMPNN mutation-effect tables, the FEP-matched MD native-contact table, and the random and ESM2-proposed variant tables — all built on the 1MEL and 4IDL scaffolds — had no exact match to any NbBench training, validation, or test sequence. The diverse-panel SAbDab MD table contained exact matches to 2

NbBench training, 4 validation, and 8 test sequences. We disclose this because it bears on the simulation-design comparison. The diverse panel gives a small frozen-encoder gain, so the overlap is a limitation of that secondary effect and is one reason not to interpret the matched-versus-heterogeneous contrast as a fully isolated causal experiment. It cannot explain the matched-scan result, whose sequences have no exact NbBench overlap.

Computational-label sampling. For mutation-effect experiments with requested count n , the loader samples up to n rows independently from each of the two template tables using the run seed; the final FEP setting with $n = 320$ therefore supplies 320 sampled 1MEL rows and 320 sampled 4IDL rows before the computational-task split. The FEP-matched MD native-contact label samples from the same two template-specific tables. The diverse-panel MD control samples n rows from its single table. Computational-label rows are split 80/20 into computational-training and computational-validation rows with the same seed. The production training call then filters the validation data passed to the trainer so that checkpoint selection and early stopping depend only on the experimental Tm validation examples.

Diverse-panel MD control: SAbDab structure selection

The following describes the *diverse-panel* MD native-contact comparison used for the simulation-design contrast in Fig. 2. The FEP-matched MD label used for the main result is built on the 1MEL/4IDL mutation scans as described in Methods. The diverse-panel MD control was built from SAbDab, the Structural Antibody Database [2]. The archived SAbDab nanobody summary table holds 2422 rows with PDB identifier, heavy- and light-chain assignments, model index, antigen annotation, experimental method, resolution, species, and other fields. The manifest workflow matches this summary against the archived IMGT-renumbered PDB files.

Structure-selection rule. The rule is deterministic. For each PDB identifier with both a SAbDab summary row and an IMGT-renumbered PDB file, the workflow takes the first heavy-chain row in the SAbDab table and accepts the chain only if that chain identifier is present in the IMGT PDB file. The output manifest holds one entry per PDB identifier, with fields for the manifest row number, PDB identifier, selected heavy chain, SAbDab TSV row, IMGT PDB path, and MD run directory. This initial one-per-PDB manifest contained 1177 structures.

400 K simulation panel. The 400 K simulations reused the systems prepared for the 300 K runs and built a 1146-entry 400 K manifest, consisting of 798 X-ray, 345 electron-microscopy, and 3 NMR structures according to the SAbDab method field. None of the selected rows had an annotated light chain, consistent with a single-domain antibody panel. After trajectory and Q-value processing, 1143 unique PDB identifiers yielded usable computational-label rows.

MD system preparation and simulation protocol

System preparation. Each structure was prepared as a nanobody-only system. The workflow takes the IMGT-renumbered PDB as the input structure, extracts the selected heavy chain, keeps only protein atoms, and leaves the termini uncapped, at pH 7.4. Chains are cleaned with PDBFixer, and pH-aware protonation is assigned through pdb2pqr/PROPKA when available [3, 4], with a fallback to AmberTools-based hydrogen assignment and Amber naming. The prepared chain is solvated in explicit OPC water with 15 Å padding and 0.15 M NaCl, and Amber topology and coordinate files are built with ff19SB and OPC [5–7].

300 K equilibration branch. The base 300 K branch uses OpenMM on CUDA devices [8]. It runs 1 ns of NVT followed by 1 ns of NPT at 300 K and 1 atm, with a 2-fs timestep and no hydrogen-mass repartitioning. Production at 300 K was generated for the broader study, but the computational label used here comes from the 400 K branch.

400 K production branch. The first 400 K relaxation stage restarts from the 300 K equilibrated state and runs 1 ns of NVT at 400 K with $C\alpha$ restraints; the second repeats 1 ns of restrained 400 K NVT. Both relaxation stages use a 2-fs timestep and no hydrogen-mass repartitioning. Production runs 100 ns of restraint-free 400 K NVT, with hydrogen-mass repartitioning, 4 amu hydrogen mass, a 4-fs timestep, and coordinate/energy output every 100 ps.

Integrator and reporting settings. In OpenMM, explicit-solvent periodic systems use PME electrostatics with a 1.0-nm nonbonded cutoff and constraints on bonds to hydrogen. NPT stages use a Monte Carlo barostat; NVT stages omit it. The integrator is the Langevin middle integrator with 1 ps^{-1} friction. The 100-ps reporting interval gives 1000 expected frames for a complete 100 ns production trajectory.

MD-derived Q-value extraction

Trajectory input requirements. The processed MD table was generated from the archived 400 K production trajectories. A system was eligible only if it had a production trajectory, a prepared reference structure, and an Amber parameter file. The amino-acid sequence was read from the prepared structure using the first $C\alpha$ atom of each residue, and sequences shorter than 50 residues were excluded.

Native-contact definition. Q-values are computed with MDTraj [9]. The topology is loaded from the Amber parameter

file, and the native reference is frame 0 of production. The calculation uses protein backbone heavy atoms. A candidate native contact must span more than three residues, have a reference-frame distance below 0.45 nm, and include at least one atom from a hydrophilic residue (Asp, Glu, Gln, Asn, Arg, Lys, or His, including Amber protonation variants). For a native-contact pair (i, j) with reference distance r_{ij}^0 and instantaneous distance r_{ij} , the per-frame contribution is

$$q_{ij} = \frac{1}{1 + \exp\{\beta(r_{ij} - \lambda r_{ij}^0)\}},$$

with $\beta = 50 \text{ nm}^{-1}$ and $\lambda = 1.8$, following the smooth native-contact form of Best-Hummer-style Q-value analyses [10]. The frame-level Q-value is the mean of q_{ij} over all selected native contacts.

Trajectory-level Q-value label. The trajectory-level label is the mean Q-value over the terminal portion of production. For trajectories with more than 300 frames, the implementation uses the larger of 300 frames and one third of the available frames; for shorter trajectories it uses all frames. For a complete 100 ns trajectory with 100-ps output, this is 333 terminal frames, roughly the final 33 ns. In the processed 400 K table, the number of frames used ranges from 66 to 333 (median 333), the number of native contacts from 28 to 728 (median 173), and the sequence length from 58 to 461 residues (median 122). Raw Q-values range from 0.0437 to 0.9994 and are scaled to [0,1] for computational-task training.

Sequence tokenization and encoder inputs

Sequences are tokenized with the ESM2 tokenizer for the selected encoder [11], using truncation and fixed-length padding to 160 residues. In the production protocol, the trainer receives all target and sampled computational-label rows for training, but its evaluation data are filtered to the experimental Tm validation rows.

Hot-encoder input path. The hot-encoder setting keeps tokenized sequences in the training data and updates the ESM2 weights. The frozen-encoder setting runs ESM2 once per target and computational-label sequence to precompute first-token embeddings, after which the optimizer updates only the regression trunk and task heads.

Neural-network architecture

The default encoder is the 8M-parameter ESM2 model, and the model uses its first-token representation as the sequence-level embedding. The shared trunk has dimensions hidden-size $\rightarrow 256 \rightarrow 128 \rightarrow 32$, with ReLU activations and dropout after the first two hidden layers. The Tm head maps the 32-dimensional representation to one scaled scalar prediction.

Target and computational heads. The Tm-only baseline is the encoder, the shared trunk, and the Tm head. Computational-label transfer models keep this target path and add computational heads. The main mutation-effect model uses separate computational heads for the two template-specific tasks. The code also implements computational-head controls, including a single shared mutation-effect head, a template-conditioned head, and a template-specific affine calibration on top of a shared head, to test whether the two template-specific computational labels need separate output calibration.

Architecture sensitivity analyses. Architecture controls include residual, latent, and mixture-style computational-fusion variants, in which the computational path produces a correction to the Tm prediction, a latent interaction between Tm and computational features, or a gate between a target-only expert and a computational-informed expert. The main reported FEP result uses the original shared-trunk architecture with separate template-specific computational heads; the other variants are kept as sensitivity analyses, not as the main claim.

Training objective, optimizer, and checkpoint selection

Each training example is a sequence, a scaled label, and a task identifier, and contributes loss only through the regression head for its task. The per-task loss is Huber loss with $\delta = 1.0$ on the scaled label. The default multi-task objective uses learned task-uncertainty weights [12]. For task k with loss L_k and learned log-scale s_k , the contribution is

$$\frac{L_k}{2 \exp(2s_k)} + s_k,$$

and the total loss sums the contributions from the tasks present in the minibatch. Fixed computational-loss weights are also implemented and were included in the candidate searches for computational classes where loss balance could shift validation performance.

Optimizer and schedule. The optimizer is AdamW. The default head/trunk learning rate is 3×10^{-4} , the default hot-encoder learning rate is 1×10^{-4} , weight decay is 0.04, dropout is 0.15, batch size is 16, warmup is 100 steps, and the

schedule is cosine decay over at most 400 epochs. Checkpoints are evaluated and saved once per epoch, and mixed precision is used when CUDA is available. In the hot-encoder setting, the trainer creates separate optimizer parameter groups for ESM2 parameters and for the freshly initialized trunk/head parameters; in the frozen-encoder setting, ESM2 parameters are non-trainable and no encoder group is used.

Checkpoint selection. Within each run, the best checkpoint has the lowest mean squared error on the experimental Tm validation rows, and early stopping monitors the same metric with patience 15 and threshold 0. Computational-task validation rows are not passed to the trainer in the production protocol. This detail is central to the comparison. Computational labels shape the learned representation through the training loss, but they do not decide which checkpoint is used for target-task prediction.

Candidate-setting selection and final evaluation

Candidate settings are selected on the experimental Tm validation split, tuning every source independently and separately for the frozen and hot encoder regimes with a two-stage search. Stage 1 fixes learning rate, dropout, and weight decay at defaults and searches the model architecture (shared trunk, residual fusion, or latent fusion) crossed with the auxiliary-head coupling (separate template-specific heads or a template-conditioned shared head). Stage 2 holds the best Stage 1 skeleton and searches, for the hot regime, the encoder learning rate (1×10^{-4} , 5×10^{-5} , 3×10^{-5} , 1×10^{-5}) and, for the frozen regime, the head learning rate (1×10^{-3} , 5×10^{-4} , 2×10^{-4}), each crossed with dropout (0.05, 0.15, 0.30) and weight decay (0.02, 0.10). This staged search covered 30 Stage 1 and 126 Stage 2 configurations for the core sources, and an additional 48 Stage 1 and 168 Stage 2 configurations for the structure-based comparators.

Seeded candidate evaluation. Each candidate is evaluated as a three-seed ensemble on the Tm validation split at $n = 320$ labels per scaffold table, and the single best configuration for each source and regime is then evaluated on the held-out test split as a five-seed ensemble. The run seed controls Python, NumPy, PyTorch, and CUDA randomness when CUDA is available, as well as target-training subsampling in the target-label scaling experiment, computational-row sampling, and computational-label train/validation splitting. The label-count curves use the selected per-source configuration while the number of computational labels is varied over 20, 80, 160, and 320.

Statistics and stored outputs

The primary metric is MAE in degrees Celsius on the 396 held-out test examples. For each condition, the five seeded models produce scaled Tm predictions for every test sequence; these are averaged in scaled-label space and inverse-transformed to degrees Celsius. Root mean squared error, coefficient of determination, and Spearman and Pearson correlations are also computed, but the figures emphasize MAE because it reads directly as an average temperature error.

Bootstrap intervals. Single-condition confidence intervals come from nonparametric bootstrap resampling over test examples. The absolute-error vector from the five-seed ensemble is resampled with replacement 10,000 times using a fixed bootstrap seed, and the reported 90% interval is the 5th to 95th percentile range of the bootstrapped MAE values. The 90% interval is used for consistency with the directional paired comparisons in the main text; the stored bootstrap summaries allow other interval levels to be recomputed. Paired comparisons use the same bootstrap indices for the candidate and reference absolute-error vectors. The reported Δ MAE is candidate minus reference, so negative values mean lower error than the reference.

Stored run summaries. Each training and evaluation call writes a structured summary recording the command-line arguments, relevant environment variables, resolved model settings, per-label-count metrics, bootstrap intervals, per-example absolute errors, and paired bootstrap summaries. Compact tables derived from these summaries feed the main computational-label comparison, the frozen-encoder controls, the computational-head controls, and the computational-combination controls.

Interpreting the label-count curves

Target-label scaling reference. The target-label scaling curve is a positive control on the pipeline. It tests whether the same model family and evaluation code recover the expected gain when more true experimental Tm labels are available. The FEP computational-label curve tests something different. It asks whether more labels from a physically related but distinct computed quantity improve the same held-out Tm endpoint, while the FEP-matched MD native-contact curve tests whether a structural stability observable responds to a matched simulation-acquisition design. The decisive comparisons are therefore not “simulation versus no simulation,” but whether the simulations cover relevant sequence changes and whether the physical quantity improves models selected with Tm validation data.

Supplementary figure data tables

The supplementary figures are generated from tabular computational-label files derived from the same run summaries as the main figures. The processed computational tables, rendered supplementary figures, and figure-generation code will be distributed with the public analysis archive.

Fig.-to-data and code mapping

The manuscript figures are regenerated from compact result summaries and computational-label tables rather than from scratch run directories. The mapping below is the first place to check when auditing or extending a figure.

Table 2 Manuscript Fig.-to-data mapping. The table identifies the processed summaries and scripts used to regenerate each display item.

Display item	Inputs	Script	Primary comparison
Fig. 1	Conceptual protocol	plot/make_outline_figures.py	Multi-task training with Tm-based model selection
Fig. 2	Simulation-plan schematic; design-by-regime MD effects; frozen label-count sweeps	plot/make_outline_figures.py	Simulation-design axis: matched local scans strengthen frozen native-contact transfer
Fig. 3	Source-by-regime map; focused FEP/MD contrasts; hot label-count sweeps	plot/make_outline_figures.py	Physical-observable axis: FEP transfers in both regimes, MD native-contact only frozen
Supplementary Figs. 1 to 5	Supplementary manifest and generated TSV tables	plot/make_supplementary_figures.py	Data-set composition, candidate-setting searches, model controls, count-sweep controls, and MD-feature controls

Generated supplementary tables. The final computational-label screen settings used in Figs. 3 and 4 and the frozen-encoder controls are recorded in generated supplementary tables, which list the selected architecture, validation-selected setting, computational-label count, validation MAE, held-out test MAE, bootstrap interval, and run-summary reference for each condition. A panel-level mapping table for all supplementary figures is included in the public analysis archive.

Supplementary figures

References

- [1] Zhang, Y. Tsuda, K., NbBench: Benchmarking language models for comprehensive nanobody tasks, , (2025).
- [2] Dunbar, J., Krawczyk, K., Leem, J., Baker, T., Fuchs, A., Georges, G., et al. SAbDab: the structural antibody database. *Nucleic Acids Research* 42, D1140–D1146 (2014). <https://doi.org/10.1093/nar/gkt1043>
- [3] Dolinsky, T. J., Nielsen, J. E., McCammon, J. A., Baker, N. A. PDB2PQR: an automated pipeline for the setup of Poisson–Boltzmann electrostatics calculations. *Nucleic Acids Research* 32, W665–W667 (2004). <https://doi.org/10.1093/nar/gkh381>
- [4] Olsson, M. H. M., Søndergaard, C. R., Rostkowski, M., Jensen, J. H. PROPKA3: consistent treatment of internal and surface residues in empirical pK_a predictions. *Journal of Chemical Theory and Computation* 7, 525–537 (2011). <https://doi.org/10.1021/ct100578z>
- [5] Case, D. A., Aktulga, H. M., Belfon, K., Cerutti, D. S., Cisneros, G. A., Cruzeiro, V. W. D., et al. AmberTools. *Journal of Chemical Information and Modeling* 63, 6183–6191 (2023). <https://doi.org/10.1021/acs.jcim.3c01153>
- [6] Tian, C., Kasavajhala, K., Belfon, K. A. A., Raguet, L., Huang, H., Migués, A. N., et al. ff19SB: amino-acid-specific protein backbone parameters trained against quantum mechanics energy surfaces in solution. *Journal of Chemical Theory and Computation* 16, 528–552 (2020). <https://doi.org/10.1021/acs.jctc.9b00591>
- [7] Izadi, S., Anandakrishnan, R., Onufriev, A. V. Building water models: a different approach. *The Journal of Physical Chemistry Letters* 5, 3863–3871 (2014). <https://doi.org/10.1021/jz501780a>
- [8] Eastman, P., Swails, J., Chodera, J. D., McGibbon, R. T., Zhao, Y., Beauchamp, K. A., et al. OpenMM 7: rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology* 13, e1005659 (2017). <https://doi.org/10.1371/journal.pcbi.1005659>
- [9] McGibbon, R. T., Beauchamp, K. A., Harrigan, M. P., Klein, C., Swails, J. M., Hernández, C. X., et al. MDTraj: a modern open library for the analysis of molecular dynamics trajectories. *Biophysical Journal* 109, 1528–1532 (2015). <https://doi.org/10.1016/j.bpj.2015.08.015>
- [10] Best, R. B., Hummer, G., Eaton, W. A. Native contacts determine protein folding mechanisms in atomistic simulations. *Proceedings of the National Academy of Sciences of the United States of America* 110, 17874–17879 (2013). <https://doi.org/10.1073/pnas.1311599110>
- [11] Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* 379, 1123–1130 (2023). <https://doi.org/10.1126/science.ade2574>
- [12] Kendall, A., Gal, Y., Cipolla, R. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* 7482–7491, IEEE/CVF (2018).

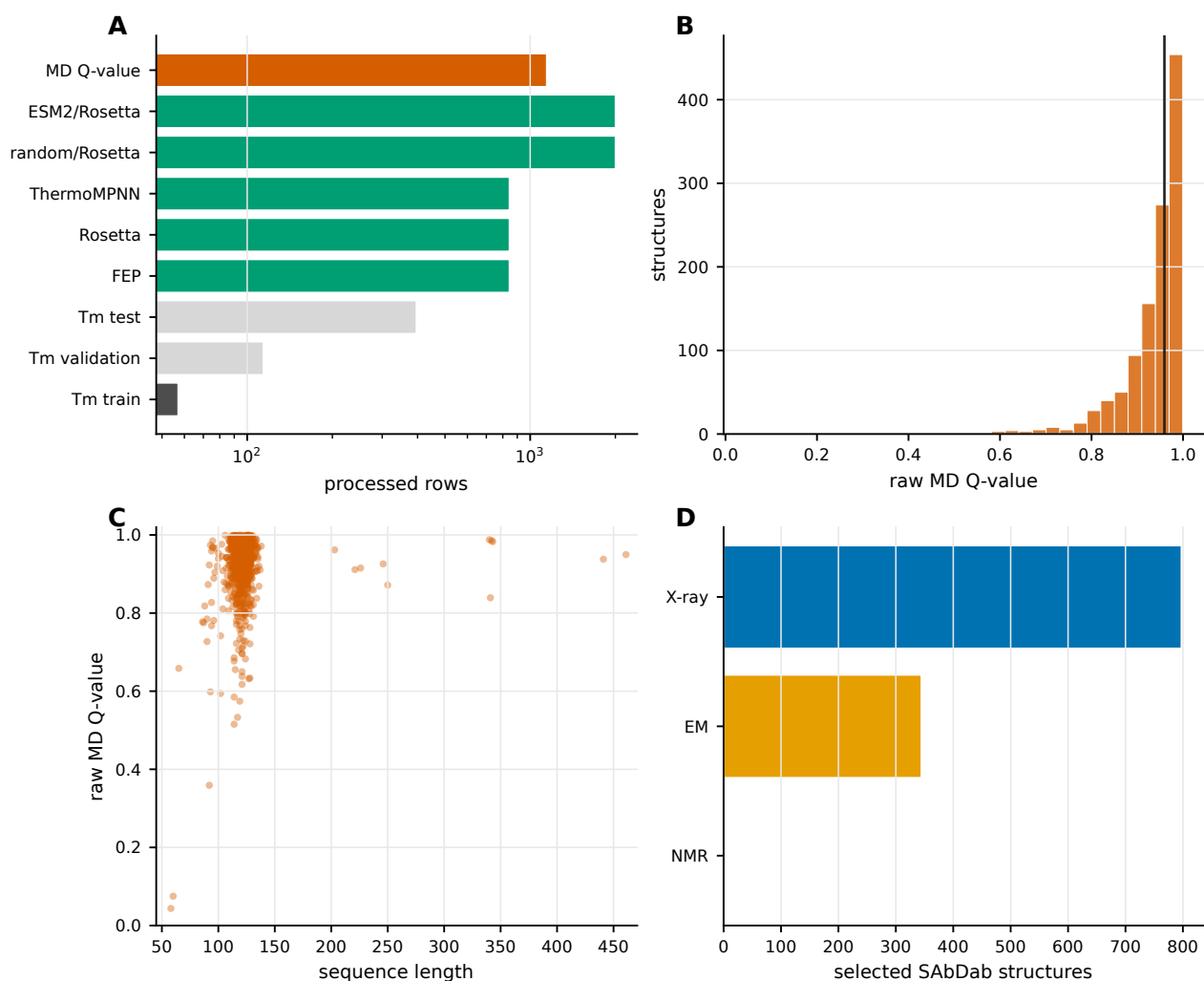


Figure 1 Processed data sets and MD-derived Q-value labels. **(a)** Processed target and computational-label row counts used in the reported experiments. **(b)** Distribution of raw MD-derived Q-values before [0,1] scaling. **(c)** Relationship between sequence length and raw MD-derived Q-value. **(d)** Experimental methods for the selected SAbDab structures used in the 400 K MD panel.

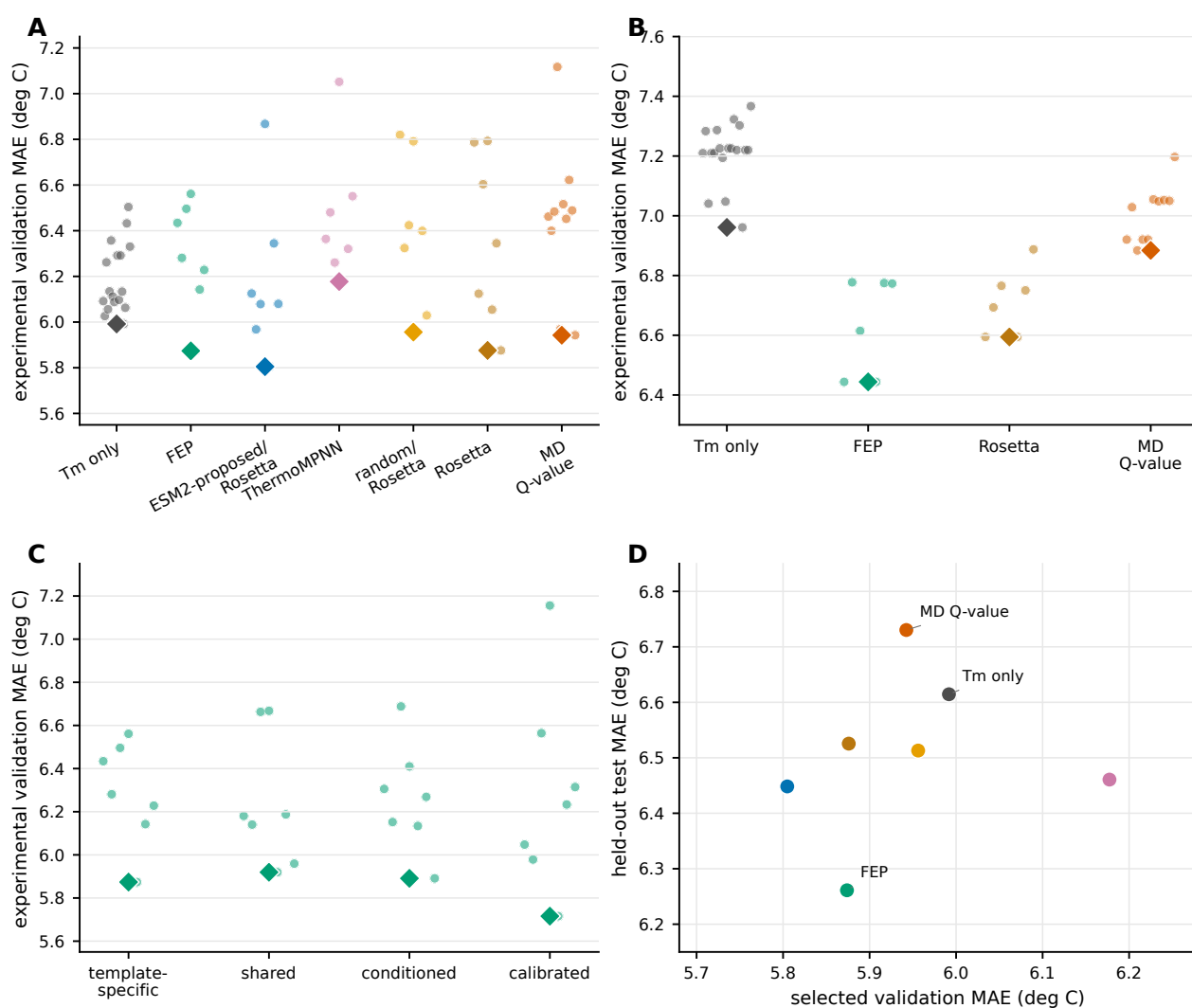


Figure 2 Candidate-setting searches and target-validation selection. **(a)** Experimental validation MAE values for hot-encoder computational-label comparisons. **(b)** Experimental validation MAE values for frozen-encoder controls. **(c)** Candidate computational-head designs for FEP mutation free-energy labels. **(d)** Relationship between selected validation MAE and held-out test MAE for the final computational-label comparison. Diamonds mark the best validation setting in panels a to c.

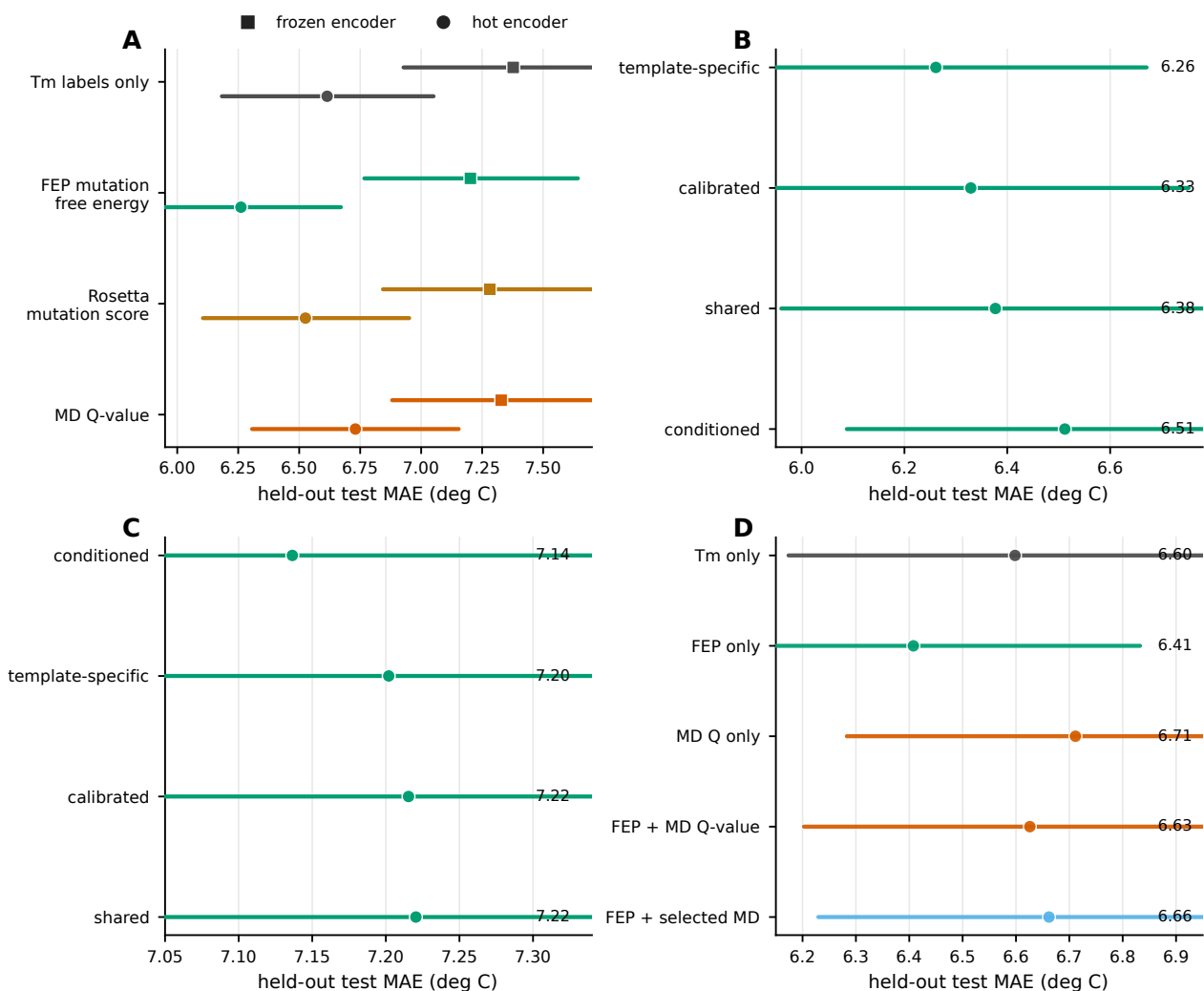


Figure 3 Model and computational-combination controls. **(a)** Frozen-encoder and hot-encoder comparisons for selected computational labels. **(b)** Hot-encoder FEP computational-head controls. **(c)** Frozen-encoder FEP computational-head controls. **(d)** Computational-combination controls comparing Tm-only, FEP-only, MD Q-value-only, and combined-computational-label settings.

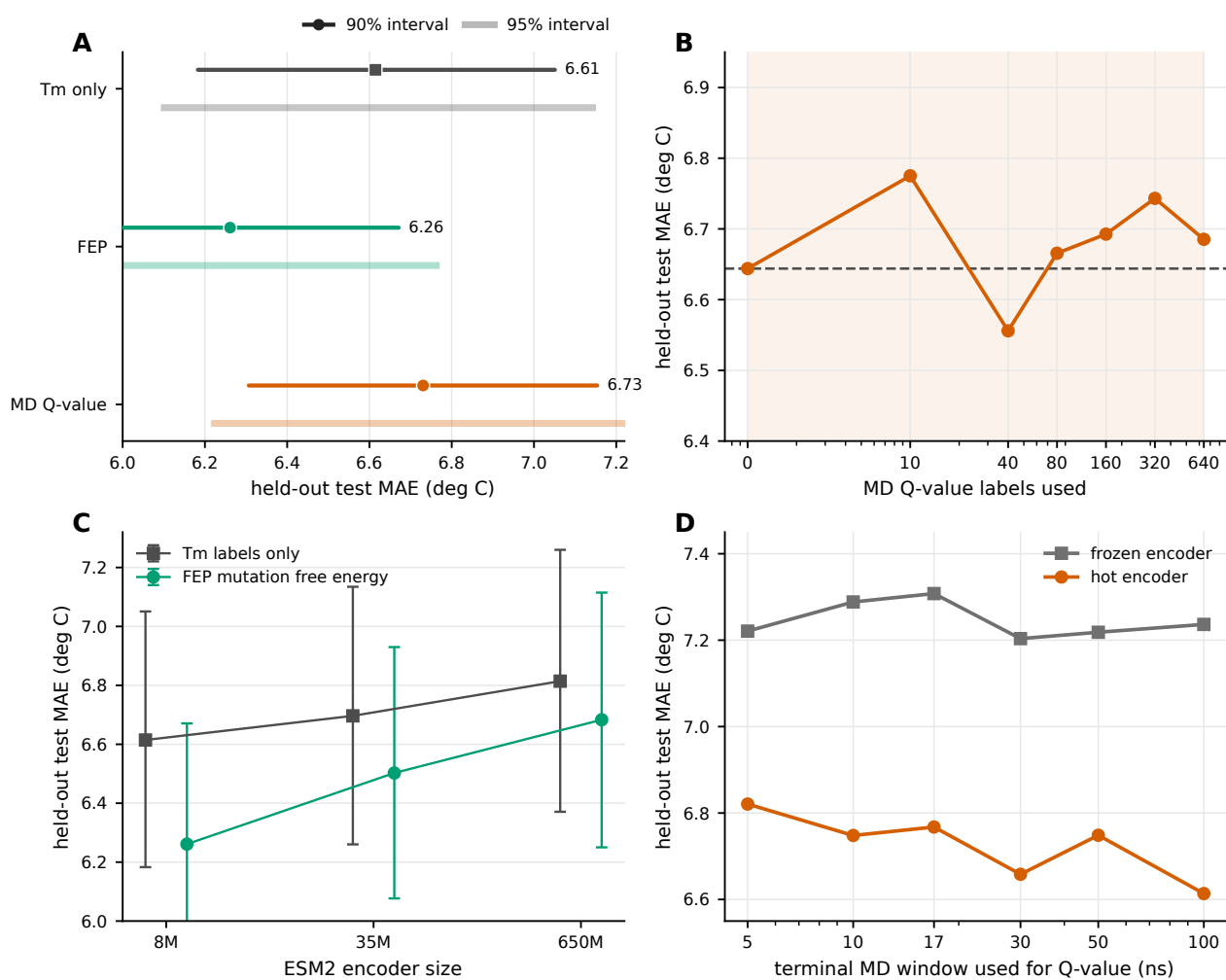


Figure 4 Interval, scaling, encoder-size, and MD-window controls. **(a)** Final Tm-only, FEP, and MD Q-value conditions shown with 90% and 95% bootstrap intervals. **(b)** MD Q-value label-count curve after selecting the candidate setting separately for each label count on the experimental validation split. **(c)** ESM2 encoder-size controls for Tm-only and FEP settings. **(d)** MD trajectory-window controls for Q-value labels.

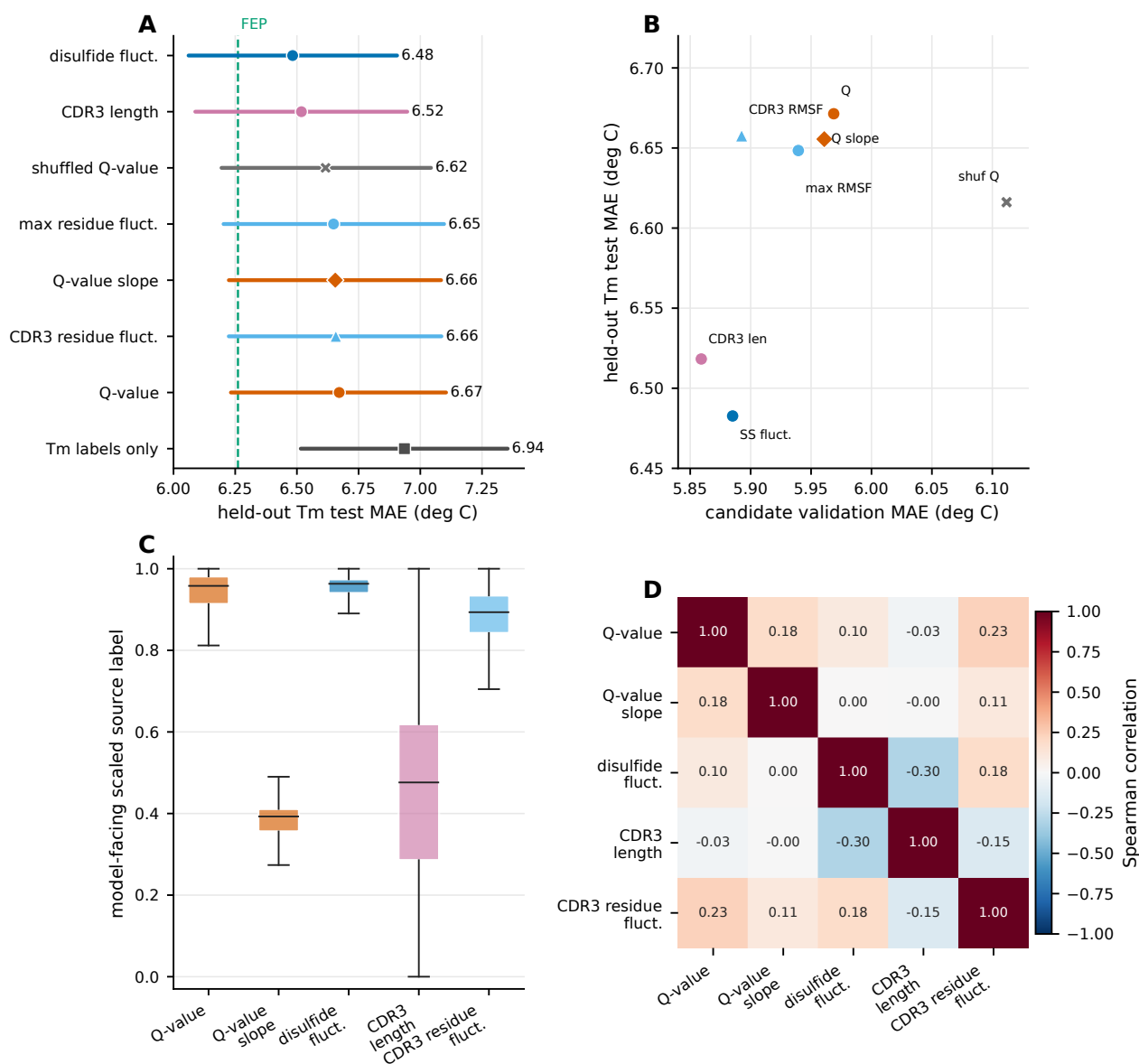


Figure 5 Descriptor controls for MD-derived computational labels. **(a)** Held-out test MAE values for selected MD-derived and nanobody-specific descriptor controls. The dashed line marks the FEP reference. **(b)** Relationship between candidate validation MAE and held-out test MAE for descriptor controls that were carried forward to final evaluation. **(c)** Distributions of the model-facing scaled computational labels for the selected descriptors. **(d)** Spearman correlations among the selected computational labels over shared sequences.